

Practical : 3	Implementation and Time analysis of Max-Heap Sort algorithm.
Date : 12/08/2026	

Objective :- To implement the Max-Heap Sort algorithm in C++ (building a max-heap and repeatedly extracting the maximum element), and to analyse its time complexity through practical execution-time measurements on inputs of varying size.

Program :-

```
#include <iostream>
#include <vector>
#include <cstdlib>
#include <ctime>
#include <chrono>
using namespace std;
using namespace std::chrono;

// ----- Max-Heapify -----
void heapify(vector<int> &arr, int n, int i) {
    int largest = i;
    int left = 2 * i + 1;
    int right = 2 * i + 2;

    if (left < n && arr[left] > arr[largest])
        largest = left;
    if (right < n && arr[right] > arr[largest])
        largest = right;

    if (largest != i) {
        swap(arr[i], arr[largest]);
        heapify(arr, n, largest);
    }
}

// ----- Build Max-Heap -----
void buildMaxHeap(vector<int> &arr) {
    int n = arr.size();
```

```
    for (int i = n / 2 - 1; i >= 0; i--)
        heapify(arr, n, i);
}

// ----- Heap Sort -----
void heapSort(vector<int> arr) {
    int n = arr.size();
    buildMaxHeap(arr);
    for (int i = n - 1; i > 0; i--) {
        swap(arr[0], arr[i]); // move current max to the end
        heapify(arr, i, 0);  // restore heap property on the reduced heap
    }
}

// ----- Timing helper -----
template <typename Func>
double measureTime(Func f, vector<int> &arr) {
    auto start = high_resolution_clock::now();
    f(arr);
    auto stop = high_resolution_clock::now();
    return duration_cast<microseconds>(stop - start).count();
}

int main() {
    srand((unsigned)time(0));
    vector<int> sizes = {100, 1000, 5000, 10000, 50000, 100000};

    cout << "Size\tHeapSort(microseconds)\n";

    for (int n : sizes) {
        vector<int> arr(n);
        for (int i = 0; i < n; i++)
            arr[i] = rand() % 1000000;

        double tHeap = measureTime(heapSort, arr);
        cout << n << "\t" << tHeap << "\n";
    }
    return 0;
}
```

Analysis:-

Time Complexity Comparison:

Algorithm	Best Case	Average Case	Worst Case	Space	Stable
Heap Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$O(n \log n)$	$O(1)$	No

Observed Execution Time (in microseconds) on a randomly generated array, alongside the theoretical $n \log_2 n$ growth for reference:

Input Size (n)	Heap Sort (μs)	$n \log_2 n$ (relative units)
100	5	664
1000	44	9966
5000	280	61439
10000	553	132877
50000	3640	780482
100000	8138	1660964

Heap Sort consists of two phases: building the max-heap in $O(n)$ time, and then performing n extract-max operations, each costing $O(\log n)$ after a swap and a re-heapify, giving an overall $O(n \log n)$ bound in the best, average, and worst case alike — unlike Quick Sort, its worst case does not degrade to $O(n^2)$. The observed timings track the $n \log_2 n$ reference column closely: as n grows from 100 to 100000 (1000x), the measured time grows by roughly 1600x, consistent with the extra $\log n$ factor ($\log_2 100000 / \log_2 100 \approx 2.5x$ on top of the 1000x growth in n). Heap Sort is not stable, since equal elements can be reordered during the swap-and-sift-down steps, but its $O(1)$ auxiliary space (it sorts in place, unlike Merge Sort's $O(n)$ buffer) makes it attractive when memory is constrained and a guaranteed $O(n \log n)$ worst case is required.

Conclusion :- Max-Heap Sort was successfully implemented and its execution time measured across increasing input sizes, confirming its theoretical $O(n \log n)$ time complexity in practice. Compared to the simple $O(n^2)$ sorts (Bubble, Selection, Insertion) studied earlier, Heap Sort scales far better for large inputs, and compared to Quick Sort it offers a guaranteed worst-case bound at the cost of typically higher constant-factor overhead and a lack of stability. Its main advantage is in-place $O(1)$ space sorting with a guaranteed $O(n \log n)$ ceiling, making it a reliable choice when worst-case performance must be bounded and auxiliary memory is limited.